Структурирани изходи (Structured Outputs)

Валидация на данни и на JSON структури

Богомил Стоянов, Михаил Доброславски

Лекция 3

Приложно AI програмиране

© 2026 Богомил Стоянов, Михаил Доброславски · CC BY-NC-ND 4.0

От Тема 2 към днес

- Тема 2: инструкции за формат на изхода („върни JSON с полета …“) работят, но са **крехки**.
- Днес ги заменяме с **гаранции**: валиден JSON, съответстващ на *схема*, който парсваме без защитен код навсякъде.
- Целта на лекцията: извличане на контакт от свободен текст → `{name, email, company, role}`, показано при трите доставчика и локално.

Тезата на лекцията

Structured outputs са **мостът** между вероятностния модел и детерминистичната програма. Целта: изходът на модела да се подава директно към следващата стъпка в кода: надеждно, типизирано, без „prompt-and-pray“.

Дневен ред: Лекция 3

- Защо структурирани изходи
- Спектър от подходи
- По доставчик
- Валидация и надеждност
- Чести грешки
- Обобщение
- Дискусия

Защо структурирани изходи

Проблемът: свободен текст не е за машинна обработка

На молбата „върни само JSON“ моделът нерядко отговаря така:

````
Разбира се! Ето извлечените данни:
```json
{ "name": "Иван Петров", "email": "ivan@fmi.example.com", }
```
Надявам се да съм бил полезен!
````

Проблеми за парсера: обяснителен текст преди/след, markdown огради ```` ```json ````, излишна запетая в края (невалиден JSON). Софтуерът се нуждае от **предвидими, типизирани** данни: полета, типове, допустими стойности от списък (enum).

„Prompt-and-pray“

Инструкция с думи + надежда, че моделът ще се подчини. Крехко: няма гаранция за (а) валиден JSON, нито за (б) съответствие със схема.

За какво ни трябват

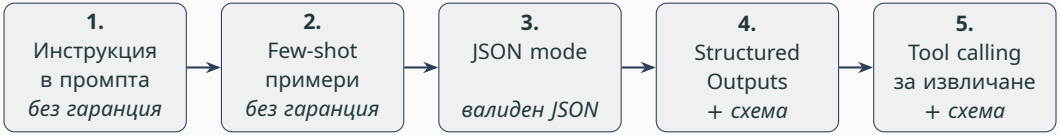
- **Извличане на данни**: неструктуриран текст → полета (визитка, фактура, CV).
- **Класификация**: фиксиран набор стойности (тон, категория, приоритет).
- **Попълване на форми / конфигурации**: директно към API, БД, UI.
- **Мост към tool calling**: структурираният изход е входът на следваща функция (Тема 9).

Целта, формулирана точно

Гаранция за **(а)** синтактично валиден JSON *и* **(б)** съответствие с *нашата* схема, за да парсваме веднъж, на едно място, без `try/except` из целия код.

Спектър от подходи

От най-крехък към най-надежден



- **(1)–(2)** само подканват („prompt-and-pray“); **никаква** гаранция.
- **(3)** JSON mode гарантира валиден JSON, но **не** схемата.
- **(4)–(5)** *constrained decoding*: моделът се ограничава при декодиране да произвежда само токени, валидни спрямо схемата (grammar). При (5) гаранцията важи за `strict` инструменти.
- (4) и (5) дават **еднаква** гаранция; (5) не е „по-силно“, а е удобно, когато моделът и без това ползва инструменти.

JSON mode vs. Structured Outputs: ключовата разлика

	JSON mode	Structured Outputs
Валиден JSON	**да**	**да**
Съответства на схема	*не*	**да**
Точни полета/типове	не гарантира	гарантира
Механизъм	само JSON синтаксис	constrained decoding по схема

Честа грешка

Бъркане на двете. JSON mode ще ви даде *някакъв* валиден JSON, но не непременно с вашите полета. За гаранция по схема ползвайте schema-constrained режим.

По доставчик

Единствен източник на истина: Pydantic модел

Дефинираме схемата **веднъж**: оттук идват и API-ограничението, и валидацията, и типовете в кода.

```python
from pydantic import BaseModel, EmailStr   # pip install "pydantic[email]"

class Contact(BaseModel):
    name: str
    email: EmailStr               # в схемата: "format": "email"
    company: str | None = None    # "по избор" = може да е null
    role: str | None = None

text = "Иван Петров, чистач във ФМИ — ivan@fmi.example.com"   # входът в примерите
```

`Contact.model_json_schema()` дава JSON схемата за подаване към API-то; `Contact.model_validate_json(raw)` валидира суров низ. Една дефиниция → три употреби. (В кода на упражнението за простота е `email: str`.)

OpenAI: Structured Outputs (parse helper)

```python
from openai import OpenAI
client = OpenAI()

resp = client.chat.completions.parse(       # helper: връща вече парснат обект
    model="gpt-6-sol",
    messages=[
        {"role": "system", "content": "Извлечи контакт от текста."},
        {"role": "user", "content": text},
    ],
    response_format=Contact,                 # Pydantic модел = схемата
)

msg = resp.choices[0].message
if msg.refusal:                              # ВИНАГИ проверявай преди .parsed
    print("Отказ:", msg.refusal)
else:
    contact: Contact = msg.parsed            # типизиран обект, готов за употреба
```

Responses API: `responses.parse(…, text_format=Contact)`.

OpenAI: ограничения на `strict` режима

- Изисква `additionalProperties: false` на всеки обект.
- **Всички** полета трябва да са в `required`; „по избор“ се прави с *union с null* (напр. `company: str | None`). Helper-ът `parse` добавя тези две неща вместо вас, затова `= None` в Pydantic модела не пречи.
- Поддържа подмножество на JSON Schema: `enum`, вложени обекти, масиви, рекурсия чрез `$ref`; коренът е обект; неподдържани конструкции се отхвърлят.
- **Refusal**: при отказ поради политиките за безопасност отговорът има поле `refusal` (а не изходната схема); проверявай преди парсване.
- Внимавай за `finish_reason="length"`: отрязан → невалиден.
- TypeScript: `zodResponseFormat(ContactSchema, "contact")` като `response_format`.

Anthropic: извличане чрез tool calling

Класическият надежден подход: дефинираш инструмент със схема и *задължаваш модела да го извика*.

```python
import anthropic
client = anthropic.Anthropic()

msg = client.messages.create(
    model="claude-haiku-4-5", max_tokens=1024,       # max_tokens е задължителен
    tools=[{"name": "extract_contact", "description": "Извлечи контакт",
            "input_schema": Contact.model_json_schema()}],
    tool_choice={"type": "tool", "name": "extract_contact"},  # задължително
    messages=[{"role": "user", "content": text}],
)

block = next(b for b in msg.content if b.type == "tool_use")
contact = Contact.model_validate(block.input)        # структурираният изход
```

`tool_use.input` е структурираният изход (подробно в Тема 9). Принуден `tool_choice` работи при Haiku 4.5, но *не* при най-новите модели (следващия слайд).

Anthropic: native structured outputs

```
msg = client.messages.parse(                # helper: връща вече валидиран обект
    model="claude-sonnet-5-5", max_tokens=1024,
    messages=[{"role": "user", "content": f"Извлечи контакт: {text}"}],
    output_format=Contact,                  # Pydantic модел = схемата
)
contact: Contact = msg.parsed_output
```

- Без beta header. Суровата форма в `messages.create` е `output_config={"format": {"type": "json_schema", "schema": …}}`; `output_format` е само удобство на `parse`.
- **Strict tool use**: `"strict": true` в дефиницията на инструмента гарантира, че входът му спазва схемата.
- Най-новите модели (Sonnet 5.5, Opus 5.5, Fable 5.1) **отхвърлят** принудителен `tool_choice` (грешка 400); там native structured outputs е правилният път.

Gemini: response_json_schema

```python
from google import genai
client = genai.Client()

resp = client.models.generate_content(
    model="gemini-3.8-flash",
    contents=text,
    config={
        "response_mime_type": "application/json",
        "response_json_schema": Contact.model_json_schema(),
    },
)
contact = Contact.model_validate_json(resp.text)   # валидираме с Pydantic
```

- `google-genai`, `generate_content` (като в Тема 1). По-старото `"response_schema"`: `Contact` + `resp.parsed` още работи и е в упражнението.
- Документацията вече води с новото Interactions API (`interactions.create`, `response_format`); идеята е същата.

Класификация с enum: фиксирани стойности

Когато изходът е *една от* няколко стойности (тон, категория):

```python
from enum import Enum
class Tone(str, Enum):
    positive = "positive"
    neutral  = "neutral"
    negative = "negative"

class Review(BaseModel):
    tone: Tone                    # в схемата: "enum": [...]

resp = client.chat.completions.parse(
    model="gpt-6-sol", response_format=Review,
    messages=[{"role": "user", "content": "Обслужването беше отвратително."}],
)
print(resp.choices[0].message.parsed.tone)   # -> Tone.negative
```

Същото `Review` работи и при Anthropic и Gemini. Enum = най-простата и най-надеждна форма на structured output.

Локален модел: Ollama (същата схема, локално)

Constrained decoding не е запазено само за облачните API-та:

```python
import ollama

resp = ollama.chat(
    model="llama3.2:1b",
    messages=[{"role": "user", "content": text}],
    format=Contact.model_json_schema(),   # схемата ограничава декодирането
    options={"temperature": 0},
)
contact = Contact.model_validate_json(resp["message"]["content"])
```

Вътрешно Ollama превръща схемата в **граматика** и с нея ограничава декодирането (идеята идва от GBNF граматиките на `llama.cpp`). Това е *същият* принцип като облачните Structured Outputs, но на вашата машина и без ключ. Пак валидираме с Pydantic (defense-in-depth).

Валидация и надеждност

Валидирай на клиента: многослойна защита

Дори при „гарантирани“ schema-constrained API-та:

- Версии на модели и режими се **менят**.
- JSON mode **не** гарантира схема; входните аргументи на инструментите също подлежат на проверка.
- Типово-валиден изход може да е **семантично** грешен (отрицателна възраст, несъществуваща валута).

Pydantic / Zod като валидатор

`Contact.model_validate_json(raw)` хвърля `ValidationError` при несъответствие. В TypeScript: `schema.safeParse(obj)` и разклонение по `success`. **Едно** определение → схема, валидация и типове.

Видове грешки

Грешка	Кога възниква / как се улавя
Невалиден JSON	слаби подходи (1–2); JSON mode го изключва
Валиден JSON, грешна схема	липсващо/грешно поле → `ValidationError`
Refusal (отказ)	`refusal` (OpenAI), `stop_reason="refusal"` (Anthropic)
Отрязан отговор	`finish_reason="length"` / `stop_reason="max_tokens"`
Халюцинирана стойност	минава схемата, но е семантично грешна

- Схемата хваща *типове*; бизнес правилата (диапазони, референтна цялост) се проверяват отделно.
- Задължително поле, а информацията липсва → моделът е *принуден* да измисли стойност. Каквото може да липсва: `| None`.
- Ниска `temperature`; логвай суровия изход при грешка.

Self-healing retry цикъл

При `ValidationError` върни грешката към модела и опитай пак:

```python
from pydantic import ValidationError

def extract(text: str, attempts: int = 3) -> Contact:
    messages = [{"role": "user", "content": text}]
    for _ in range(attempts):
        raw = call_model(messages)                  # връща суров JSON низ
        try:
            return Contact.model_validate_json(raw)
        except ValidationError as e:
            messages += [{"role": "assistant", "content": raw},
                         {"role": "user",
                          "content": f"Изходът не мина валидация: {e}. Поправи го."}]
    raise RuntimeError("Изчерпани опити; не приемаме невалидни данни.")
```

`call_model`: произволен доставчик. Таван на опитите + ясен fallback; **никога** тихо приемане на невалидни данни. При schema-constrained режим цикълът хваща главно *бизнес* правила (Pydantic валидатори).

Чести грешки

Чести грешки

- **JSON mode ≠ Structured Outputs**: първото не гарантира полетата.
- **Ръчно парсване** с regex/`json.loads`: крехко; ползвай API механизмите.
- **Липсваща проверка за отказ** (`refusal`) преди парсване.
- **Пропуснати strict изисквания**: `additionalProperties`, пълен `required`.
- **Без клиентска валидация**: API гаранциите се менят; валидирай винаги.
- **Схема ≠ бизнес правила**: типово валидно ≠ семантично вярно.
- **Нисък `max_tokens`** (Anthropic) → отрязан, невалиден JSON.
- **Две дефиниции на схемата**: рано или късно се разминават.

Обобщение

Какво научихме днес

- Structured outputs свързват вероятностния модел с детерминистичния код.
- Спектър: инструкция → few-shot → JSON mode → **schema-constrained** → tool calling; гаранцията идва от constrained decoding.
- OpenAI: `parse` + `strict`; Anthropic: `messages.parse` или tool calling; Gemini: `response_json_schema`; Ollama: `format`.
- **Единствен източник на истина** (Pydantic/Zod) → схема, валидация, типове.
- Надеждност: `refusal`, клиентска валидация, self-healing retry, семантични проверки.

Следваща лекция

Тема 4 (UI/UX и streaming): частичен структуриран изход в интерфейса.

Връзка с изпитния конспект

- **Въпрос 4**: „Техники за извличане на структурирани данни (Structured Outputs) от недетерминирани модели.“ → покрит от цялата тема: спектъра от подходи, механизмите по доставчик, валидацията и retry.
- **Косвено Въпрос 3**: промпт-инструкциите за формат (роли/контекст, Тема 2) тук се заменят с гарантирани схеми.
- **Мост към tool calling**: structured output ↔ function calling; механизмът на схема на входа се въвежда тук, разгръща се в Тема 9.

Дискусия

Теми

1. Кой носи отговорност за валидността на данните: моделът, API-то (JSON mode) или нашият валидатор? На кого се доверявате най-малко?
2. Какво правим, когато моделът върне изход, валиден *по схема*, но семантично грешен?
3. Ограничават ли структурираните изходи качеството/креативността на модела? Кога това е приемлив компромис?

Библиография

- Alammar & Grootendorst (2024). *Hands-On Large Language Models*.
- OpenAI Structured Outputs: `https://developers.openai.com/api/docs/guides/structured-outputs`
- Anthropic Tool use: `https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview`
- Anthropic Structured outputs: `https://platform.claude.com/docs/en/build-with-claude/structured-outputs`
- Gemini Structured output: `https://ai.google.dev/gemini-api/docs/structured-output`
- Pydantic: `https://docs.pydantic.dev/` Zod: `https://zod.dev/`
- JSON Schema: `https://json-schema.org/`